

C++ Programming: Problem Solving and Programming

Chapter 6 Advanced Function II

Objective

- Passing of arrays
- Passing of structures
- Storage classes
- Scope
- Duration

Introduction

- Passing of arrays and its elements as well as passing of structures and its members to and from the functions can be done via the methods of pass by value, reference and address
- It allows the functions to process more complex data types efficiently

Passing Array to Function

Arrays can be passed to function in two ways:

1. Individual elements
(Pass by **value**)
2. Whole array
(Pass by **reference**)

Note: Returning an array is NOT allowed

Passing Array to Function (Individual element)

- Using **pass-by-value** – similar to passing values from actual parameters to formal parameters.

- Example:

```
void initialize (int x)
{
    . . . . ;
}
```

- Function call:

```
initialize (list[5]) ;
```

Passing Array to Function (Individual element) - Example

```
#include <iostream>
#define SIZE 15
using namespace std;
int findLargest(int, int);
```

```
int main(){
    int list[SIZE] = { 5, 2, 7, 8, 36, 4, 2, 68, 22, 34, 11 };
    int largest, i;

    for (largest = list[0], i = 1; i<SIZE; i++)
        largest = findLargest(list[i], largest);
    cout << "The largest number is " << largest << endl;
    return 0;
}
```

Pass an individual
element of the array

```
int findLargest(int x, int largest){
    if (x > largest)
        return x;
    else
        return largest;
}
```

Normal variable declared
as formal parameter to
receive value from the
array element

Passing Array to Function (Individual element) - Exercise

- Write a C++ function to convert a temperature values in Celsius to Fahrenheit. It will receive a float type value in Celsius and return the equivalent value in Fahrenheit.
- The conversion formula is
$$\text{Fahrenheit} = (\text{Celsius} * 9.0 / 5.0) + 32;$$
- The main function should declare an array to store 5 temperature values in Celsius. It will perform the conversion of these values to Fahrenheit using the above function and display the result.

Passing Array to Function (Whole array)

- Using **pass-by-reference** – **Do not** use the symbol **&** when declaring an array as a formal parameter.
- The **size** of the array is usually omitted. If the size is specified when it is declared as a formal parameter, the size is ignored by the compiler.

- Example:

No **&** symbol and the **size** is not specified

```
void initialize (int list[], int size)
{
    ....;
}
```

Passing Array to Function (Whole array) - Example

```
#include <iostream>
#define SIZE 15
using namespace std;
int findLargest(int []);
```

```
int main() {
    int list[SIZE] = { 5, 2, 7, 8, 36, 4, 2, 68, 22, 34, 11 };
    int largest;
    largest = findLargest(list);
    cout << "The largest number is " << largest ;
    return 0;
}
```

pass only the name of the array

```
int findLargest (int x[]) {
    int largest, i;
    for (largest = x[0], i = 1; i < SIZE; i++)
        if (x[i] > largest)
            largest = x[i];
    return largest;
}
```

formal parameter of the
function definition must be
an array type (use [])

size of the array does not
need to be specified

Passing Array to Function (Whole array) - Example

```
#include <iostream>
#define SIZE 5
using namespace std;
void multiply2(int x[], int size);
int main()
{
    int i, base[SIZE] = { 3, 7, 1, 5, 8 };

    cout << "Before function call : \n";
    for (i = 0; i < 5; i++)
        cout << base[i] << " ";
    cout << endl << endl;

    multiply2(base, SIZE);

    cout << "After function call : " << endl;
    for (i = 0; i < 5; i++)
        cout << base[i] << " ";
    cout << endl << endl;
    return 0;
}
```

```
void multiply2(int x[], int size)
{
    int i;
    cout << "Printed from multiply2\n";
    for (i = 0; i < size; i++)
    {
        x[i] *= 2;
        cout << x[i] << " ";
    }
    cout << endl << endl;
}
```

Passing Array to Function (Whole array) - Example

. Output:

Before function call:

3 7 1 5 8

Printed from multiply2 :

6 14 2 10 16

After function call:

6 14 2 10 16

The original values changed after function call.

WHY? —————→ **Array name** is actually a **pointer**

Passing Array to Function (Whole array) - Exercise

- Write a C++ function to find the number of students who passed an exam of the Civics subject where a list of student marks will be passed to the function. It will return the number of students. The passing mark is 50.
- Build the main function where a list of marks are initialized into a mark array of a class which has 25 students. Then it will display the number of students of passed the Civics course.

Passing strings to functions

```
void fun1(char x[])  
{  
    cout << x;  
}
```

```
void fun2(char y[])  
{  
    strcpy(y, "love");  
    cout << y;  
}
```

```
void main()  
{  
    char str[5] = "hate";  
    fun1(str); ①  
    fun2(str); ②  
    cout << str; ③  
}
```

① hate

② love

③ love

Passing the Two Dimensional array (Individual element)

```
void display( int x )  
{  
    cout << x;  
}
```

```
void main()  
{  
    int num[3][2]={ {11,22}, {33,44}, {55,66} };  
    display(num[1][2]);  
}
```

Passing the whole Two Dimensional array

```
void display( int x[][] )
{
    for( .. )
        for( .. )
            cout << .. // display all numbers
}
```

```
void main()
{
    int num[3][2]={ {11,22}, {33,44}, {55,66} };
    display(num);
}
```

Passing Structure to Function

A structure can be passed to function in three ways:

1. Individual members
2. Whole structure
(Pass by **value / input parameter**)
3. Address of a structure or member
(Pass by **reference / output parameter**)

Note: Returning a structure is allowed

Passing individual members of structure (pass by value)

```
void update( float cgpa ) {  
    cgpa = 4.0; cout << cgpa;  
}
```

CGPA is changed, but not stud.cgpa

*Input is passed by **value***

```
void main() {  
    struct Student {  
        char name[30], id[12];  
        float cgpa;  
    } ;
```

Output 1?

Output 2?

```
Student stud = {"Andy", "12AA11", 3.5};  
update(stud.cgpa); ①  
cout << stud.cgpa; } ②
```

Passing the whole structure (pass by value)

```
struct Student{  
    char name[30], id[12]; float cgpa;} ;
```

```
void update( Student s )  
{  
    s.cgpa = 4.0; cout << s.cgpa;  
}
```

Output 1?

```
void main()  
{ Student stud = {"Andy", "12AA11", 3.5};  
  update(stud); ①  
  cout << stud.cgpa; ②  
}
```

Output 2?

Passing Whole Structure to Function (pass by value) - Exercise

- Write a C++ function to display a student's name, ID and CGPA where the info is received in a student structure
- The main function should define a structure of Student which carries the student's name, ID and CPGA. Declare a variable for this structure and initialize some relevant values. The info of the students will be displayed accordingly after invoking the function above.

Passing the whole structure (pass by reference/address)

```
struct Student{  
    char name[30], id[12]; float cgpa;  
};
```

```
void update( Student *s ) {  
    s->cgpa = 4.0; cout << s->cgpa;  
}
```

```
void main()  
{ Student stud = {"Andy", "12AA11", 3.5};  
  update(&stud); ①  
  cout << stud.cgpa; ②  
}
```

Output 2?

Exercise (Pass the whole structure by reference)

```
struct Student {  
    char name[30]; float cgpa;};
```

```
void askRecord( (A) &s )  
{  
    cin >> (B); //get name  
    cin >> (C); //get cgpa  
}
```

```
void main()  
{ Student stud;  
    askRecord((D));  
    cout << stud.name << stud.cgpa);  
}
```

Passing Whole Structure to Function (pass by reference & address) - Exercise

- In a car maintenance tracker for a workshop, define a structure Car to store the info below:
 - model (a string): the name of the car model (e.g., "Myvi", "Vios").
 - km (an integer): total kilometers a car travelled
 - fuel (a double): amount of fuel in car's tank
- Create functions below:
 - serviceCar takes a car by reference and reset the km to zero
 - refuel takes a pointer to car and number of liters which will be added to the current fuel amount

Passing Whole Structure to Function (pass by reference & address) - Exercise

- A main function to define the Car structure and declare its variable car with initial values of “X70”, 56000, and 18.5
 - Call the serviceCar and refuel functions
 - Display the updated car details

Returning a Structure

```
struct Student{  
    char name[30]; float cgpa;  };  
  
Student askRecord()  
{ Student stud;  
    cin >> stud.name;  
    cin >> stud.cgpa;  
    return stud; //structure can be returned  
}  
  
void main()  
{ Student stud2 = askRecord();  
    cout << stud2.name << stud2.cgpa;  
}
```

Returning a structure - Exercise

- Write a C++ function to get user to enter the current time in hour and minute. The info is stored in a variable of Time structure which will be returned to the calling function.
- The main function defines a structure of Time which consists of hour and time (both in integer). It will invoke the function to input the time info then displaying them.

Storage Classes

- Define the characteristics of variables and functions, specifically their scope (visibility), lifespan (duration) and memory location.
- Understanding storage classes is crucial for managing variable behavior and optimizing program performance.

Storage Classes of Variable

Storage class of a variable determines its visibility, duration and location

Visibility

- A program's ability to access that variable's memory location

Duration

- The time during which a memory location exists for that variable

Location

- Where it is located in the main memory at run time

Storage Classes of Variable

Scope - The region in the program in which a variable can be used

Variables are in scope from their point of declaration until the end of their block

```
#include <stdio.h>
void fun(int a, int b);

void main() {
    int a=0, b, c;
    if(a>0)
    {
        float x, y;
        x = a * 2;
    } //end of nested block
} //End of main block

void fun(int a, int b) {
    cout << a << b;
}
```

Global area

main's area

nested block area

fun's area

These specific *a*, *b* are within the fun's area scope only

Storage Classes of Variable

Example

```
1. int a=10;
2.
3. void show(int b)
4. {    printf("%d", a);
5.     int a=20;
6.     printf("%d", a);
7. }
8.
9. int d;
10. void main()
11. {
12.     int e;
13.     show(e);
14. }
```

variable	scope
a (line 1)	1-14
b (line 3)	3-7
a (line 5)	5-7
d (line 9)	9-14
e (line 12)	12-14

Storage Classes of Variable

E
x
e
r
c
i
s
e

```
1.  int x=10;
2.  void fun(int x)
3.  {
4.      int a;
5.
6.  }
7.  int a;
8.  void main()
9.  {
10.     int x;
11.     fun(x);
12. }
```

variable	scope
x (line 1)	1-12
x (line 2)	2-6
a (line 4)	4-6
a (line 7)	7-12
x (line 10)	10-12

Storage Classes of Variable

There are FOUR key storage classes for variables:-

1. auto
2. static
3. register
4. Extern

Syntax: storage_class data type variable_name;

Example:

static

int

x;

Storage Classes of Variable

Storage class : **auto**

- Variables that are stored on the runtime stack.
- The default storage class for local variables and formal parameters.
- They are automatically allocated on the stack when a function is called and automatically de-allocated when the function returns.

Example:

```
int a;  
float b; }
```



```
auto int a;  
auto float b;
```

Storage Classes of Variable

Storage class : **auto**

- *Example:*

```
#include <iostream>
using namespace std;
void main() {
    auto int a = 100;
    cout << a;
}
```

- The variable *a* lives during the block execution, it is destroyed and ended at the closing brace.

Storage Classes of Variable

Storage class : **static**

- Space for static local variables are allocated and initialized once, conceptually at compile time.
- They are persistent variables, i.e. they are not destroyed when their function terminates. This enables a local variable to retain its value between function calls.
- They remain allocated until the entire program terminates.

Example:

```
static int a;  
static float b;
```

Value in static variable will always remain the updated values.

Storage Classes of Variable

Storage class : **static**

- *Example:*

```
#include <iostream>
using namespace std;
```

```
int counter () {
    static int count = 0;
    return count++;
}
```

In second function call to counter(), the *count* value used is 1 (retained from previous call), instead of using the initial value 0.

```
void main() {
    cout << counter(); // 1
    cout << counter(); // 2
}
```

Storage Classes of Variable

Storage class : **register**

- The compiler will attempt to store register variables in the CPU registers, thus speeding up execution and making the machine code more compact.
- But compiler will ignore its use normally

Example:

```
int a;  
register int i;
```



Store inside **Memory**



Store inside **Register**

Storage Classes of Variable

Storage class : **extern**

- Any variable defined outside a function is **external** by default. Such a variable is available without declaration to any function that appears after its definition within the same source file.
- It is also available to any function in the entire program that declares the variable using the keyword **extern**. Thus, it is sometimes called a **global variable**.
- A declaration beginning with the keyword **extern** does not allocate memory, it simply provides type of information to the compiler.

Storage Classes of Variable

Storage class : **extern**

- *Example:*

```
int b = 10; //global
void main(){
    cout << b;
    display();
}
void display(){
    int b=5; //local
    cout << b; }
```

Output:
10 5

```
int b = 10; //global
void main(){
    cout << b;
    display();
}
void display(){
    extern int b; //global
    cout << b; }
```

Output:
10 10

Storage Classes of Variable

- Exercise

```
void f1(int);
void f2(int);
int A = 20;
void main( ) {
    int B;
    for (B=0 ; B<3; B++){
        f1(B);
        f2(B);
    }
}
```

```
void f1(int E) {
    int C =2;
    C += E + A;
    cout << C << " ";
}
void f2(int F) {
    static int D = 2;
    D += F + A;
    cout << D << " ";
}
```

a) Identify the storage class for A, B, C, D and E

a) Trace the output

Storage Classes

– Duration of Variables

Duration/lifespan of a variable is the time during which a memory location exists for the variable at run time

Static Duration (global variable)

Exists when the program execution begins until the termination of the program.

Local Duration (automatic variable)

Exists when the function begins and until the end of the function

Dynamic Duration (dynamic variable)

Variables that are created and destroyed dynamically by using special functions at run time

Storage Classes

– Duration of Variables

Example

```
int num=20;
void display()
{
    int a = 10;
    cout << a;    //10
}
```

```
void main()
{
    cout << num;
    display();
    cout << num;
}
```

Storage Classes

Question

Which of the following storage class might NOT allocate memory for the variable p . Check all that apply.

- ☐ (a) int p;
- ☐ (b) auto int p;
- ☐ ☒ (c) static int p;
- ☐ ☒ (d) register int p;
- ☐ (e) extern int p;

Storage Classes

Question

Which of the following variable will be **deallocated** when the function **fun()** is terminated? Check all that apply.

☐ ☐☐ ☐☐ ☐☐ d☐ e

```
void fun() {  
    int a;  
    auto int b;  
    static int c;  
    register int d;  
    extern int e;  
}
```

```
void main() {  
    fun();  
}
```